

Docker in EC2

Class no:1

Docker:

Docker is an open-source platform that automates the deployment of applications inside lightweight, portable folders called **containers**.

Docker does the same for software. It packs your application, code, runtime, system tools, and libraries into one standard "box" so it runs exactly the same way on your laptop, a testing server, or a cloud provider.

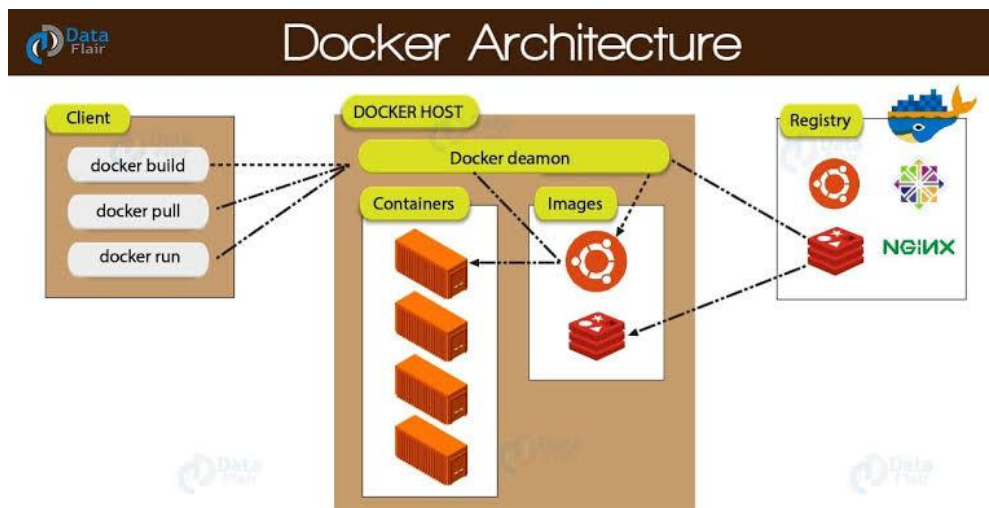
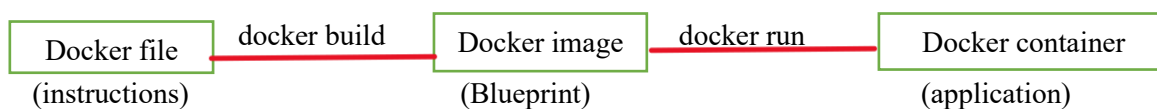
The Core Problem It Solves: It eliminates the classic developer excuse: *"Well, it worked on my machine!"*

Why Use Docker over Virtual Machines (VMs)?

- **Virtual Machines:** Each VM includes a full copy of an operating system (OS), taking up gigabytes of space and taking minutes to boot up.
- **Docker Containers:** Share the host computer's OS kernel instead of installing a whole new one. They are incredibly lightweight (often just megabytes) and spin up in seconds.

Docker Architecture

Docker uses a client-server architecture. The Docker **Client** talks to the Docker **Daemon** (the host server), which does the heavy lifting of building, running, and distributing your containers.



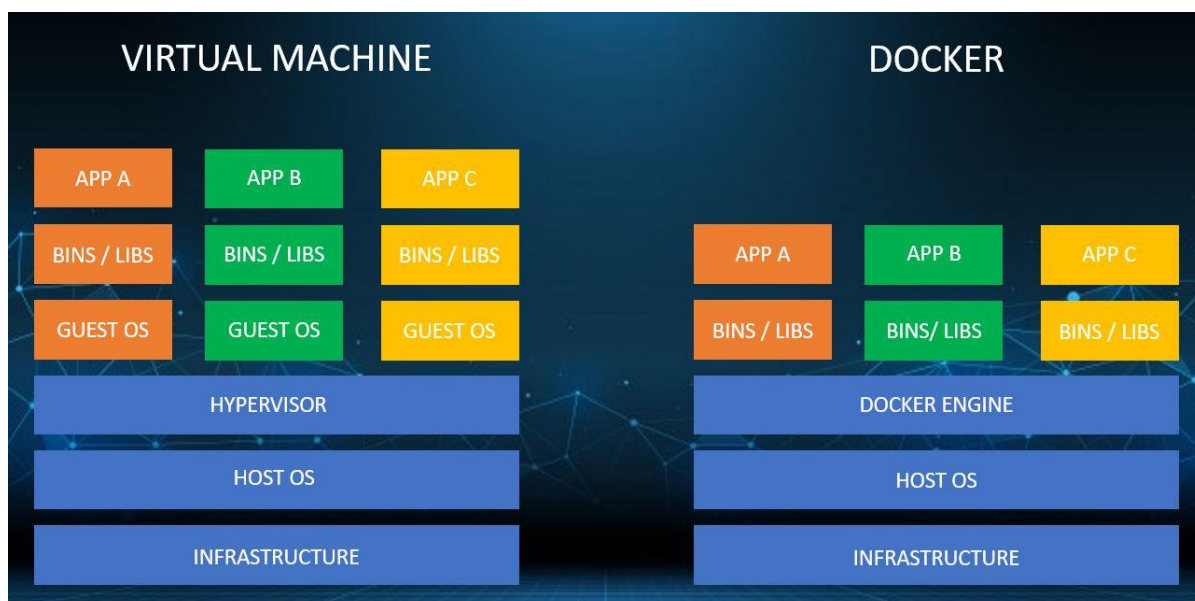
The Core Components

- **Docker file:** A simple text file containing a list of step-by-step instructions or commands used to build a Docker image. Think of it as a **blueprint or recipe**.
- **Docker Image:** A read-only, static template used to create containers. It contains the application code, dependencies, and environment configurations. If a Dockerfile is the recipe, the Image is the **pre-baked cake mix**.
- **Docker Container:** A live, running instance of an image. It is completely isolated from other containers and the host machine. This is the **actual cake** you can eat. You can start, stop, move, or delete a container.
- **Docker Engine** is a lightweight, client-server application that builds and runs containers. It is the heart of Docker.
- **Docker Registry (Docker Hub):** A centralized storage place for sharing and downloading Docker images. Docker Hub is the default public registry (similar to GitHub, but for Docker images). You can download ready-to-use images for databases (like MongoDB), web servers (like Nginx), or languages (like Python).

Summary Cheat Sheet

Component	What it is	Analogy
Dockerfile	Text configuration file	The Blueprint / Recipe
Docker Image	Read-only static snapshot	The Packaged Cake Mix
Docker Container	Live, isolated running application	The Finished Cake
Docker Daemon	The background engine managing everything	The Baker / Factory Worker
Docker Hub	Public cloud storage for images	The Grocery Store Shelf

Docker vs Virtual machine



Hands on:

Step no:1 Here our infrastructure in an EC2 instance.

Launch an instance,

- Name: **Docker-instance**
- Instance type: **Choose large**
- Security group: **Default sg with all TCP enabled**
- Storage: **20 GB**

Launch instance.

Launch an instance

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags

Name

Docker-instance

Add additional tags

Application and OS Images (Amazon Machine Image)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI

ami-09ed39e30153c3bf9 [64-bit (x86), uefi-preferred] / ami-071cfab6869e5b8bb [64-bit (Arm), uefi]

Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Description

Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.11.20260514.0 x86_64 HVM kernel-6.1

Architecture

64-bit (x86)

Boot mode

uefi-preferred

AMI ID

ami-09ed39e30153c3bf9

Publish Date

2026-05-15

Username

ec2-user

Verified provider

Instance type

Info | Get advice

Instance type

c7i-flex.large

Family: c7i-flex 2 vCPU 4 GiB Memory Current generation: true

On-Demand Ubuntu Pro base pricing: 0.08829 USD per Hour On-Demand RHEL base pricing: 0.11359 USD per Hour

On-Demand SUSE base pricing: 0.14109 USD per Hour On-Demand Linux base pricing: 0.08479 USD per Hour

On-Demand Windows base pricing: 0.17219 USD per Hour

Free tier eligible

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

Key pair (login)

Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key before you launch the instance.

Key pair name - required

KeyName

Create new key pair

Network settings

Info

Edit

Network

vpc-0538c231a226fe3c9

Subnet

No preference (Default subnet in any availability zone)

Auto-assign public IP

Enable

Firewall (security groups)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

Common security groups

Select security groups

default sg-06d80e515c32b9515

VPC: vpc-0538c231a226fe3c9

Compare security group rules

Security groups that you add or remove here will be added to or removed from all your network interfaces.

Configure storage

Info

Advanced

1x

20

GiB

gp3

Root volume, 3000 IOPS, Not encrypted

Add new volume

Instances (1) [Info](#) Last updated less than a minute ago [Connect](#) [I](#)

Saved filter sets
Choose filter set ▾

Find Instance by attribute or tag (case-sensitive)

☐	Name ↗	Instance ID	Instance state	Instance type	Status check	Availability Zone
☐	Docker-instance	i-0801e8c2a5c7349d0	Running 🔍 🔍	c7i-flex.large	Initializing	ap-south-1a

Inbound rules (7) [Manage tags](#)

Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
☐	-	sgr-0a5d99428b88b0bff	IPv4	MYSQL/Aurora	TCP	3306	0.0.0.0/0
☐	-	sgr-0ff6791321f332304	IPv4	HTTPS	TCP	443	0.0.0.0/0
☐	-	sgr-06b9fe308e8427a99	-	All traffic	All	All	sg-06d80e515c32b951...
☐	-	sgr-08197f521eb29746a	IPv4	SSH	TCP	22	0.0.0.0/0
☐	-	sgr-0e4b541e985a71540	IPv4	All ICMP - IPv4	ICMP	All	0.0.0.0/0
☐	-	sgr-0f84d83f6745d9a22	IPv4	HTTP	TCP	80	0.0.0.0/0
☐	-	sgr-02b0bb72fe6f5dc94	IPv4	All TCP	TCP	0 - 65535	0.0.0.0/0

Step no:2 Once the instance has been running connect it. Switch to root user and update the instance.

```
[ec2-user@ip-172-31-45-201 ~]$ sudo su
[root@ip-172-31-45-201 ec2-user]# yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
```

Once updated, install docker: **yum install docker**

```
[root@ip-172-31-45-201 ec2-user]# yum install docker
Last metadata expiration check: 0:00:40 ago on Fri May 22 11:47:19 2026.
Dependencies resolved.

=====
Package                                                    Architecture
=====
Installing:
docker                                                    x86_64
Installing dependencies:
container-selinux                                         noarch
containerd                                               x86_64
=====
```

Start docker and check status: **systemctl start docker , systemctl status docker**

```
[root@ip-172-31-45-201 ec2-user]# systemctl start docker
[root@ip-172-31-45-201 ec2-user]# systemctl status docker
• docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-05-22 11:48:59 UTC; 1min 1s ago
   TriggeredBy: • docker.socket
   Docs: https://docs.docker.com
```

Enable docker: **systemctl enable docker**

```
[root@ip-172-31-45-201 ec2-user]# sudo su
[root@ip-172-31-45-201 ec2-user]# systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/d
[root@ip-172-31-45-201 ec2-user]#
```

Step no:3 To list all the docker images available use : **docker images**

As there is no images, we are going to pull a new image httpd using : **docker pull httpd**

By default, it take the latest version.

```
[root@ip-172-31-45-201 ec2-user]# docker images
REPOSITORY      TAG          IMAGE ID      CREATED      SIZE
[root@ip-172-31-45-201 ec2-user]# docker pull httpd
Using default tag: latest
latest: Pulling from library/httpd
5b4d6ff92fc4: Pull complete
f086d7c0b862: Pull complete
4f4fb700ef54: Pull complete
f60b6f23733a: Pull complete
c3f994ae0cac: Pull complete
ed804d184361: Pull complete
Digest: sha256:2a7deaaaf357261a1dffc8b8fc725f4aaba2af95fbc
Status: Downloaded newer image for httpd:latest
```

Once downloaded, we can see the images.

```
[root@ip-172-31-45-201 ec2-user]# docker images
REPOSITORY      TAG          IMAGE ID      CREATED      SIZE
httpd            latest      b9097c67d4a2  2 days ago  117MB
[root@ip-172-31-45-201 ec2-user]#
```

Step no:4 We can run the image using :

docker run -itd --name containername -p host_port:container_port image_name:tag

It d → interactive terminal detached

-p → port number

```
[root@ip-172-31-45-201 ec2-user]# docker run -itd --name container1 -p 8080:80 httpd:latest
e67af5af8ac196650fe44b33c051d77efc8254ddb148102de4aaa546afefd303
```

To see the list of running containers: **docker ps**

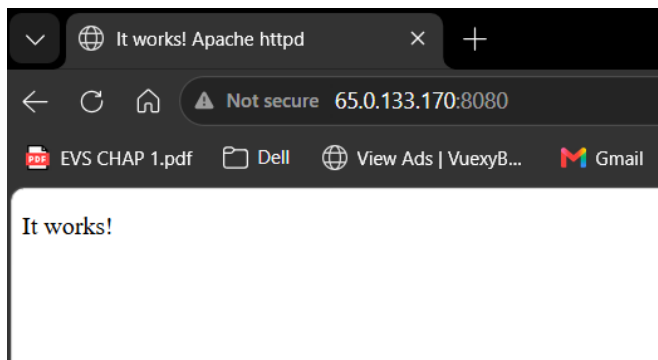
To see the list of all containers in different stages like stop, running and unused status: **docker ps -a**

```
[root@ip-172-31-45-201 ec2-user]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED
e67af5af8ac1   httpd:latest   "httpd-foreground"      15 seco
[root@ip-172-31-45-201 ec2-user]# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED
e67af5af8ac1   httpd:latest   "httpd-foreground"      20 seco
```

Step no:5 Copy the public ip , paste it followed by 8080 (host port). We can see the httpd output.

```
[root@ip-172-31-45-201 ec2-user]#
```

i-0801e8c2a5c7349d0 (Docker-instance)
PublicIPs: 65.0.133.170 PrivateIPs: 172.31.45.201



Step no:6 Now I am going to pull **nginx** image

```
[root@ip-172-31-45-201 ec2-user]# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
5b4d6ff92fc4: Already exists
cf902b7062fb: Pull complete
564041453b75: Pull complete
fa14110e5ccb: Pull complete
e2dc6cdcd988: Pull complete
5735e617770d: Pull complete
888081780287: Pull complete
Digest: sha256:800e7c98538c6bf725f5177e841aa720ae0ed1c
Status: Downloaded newer image for nginx:latest
```

```
[root@ip-172-31-45-201 ec2-user]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest   93165d051206   2 days ago    161MB
httpd         latest   b9097c67d4a2   2 days ago    117MB
[root@ip-172-31-45-201 ec2-user]#
```

Similarly, run the image using **docker run**

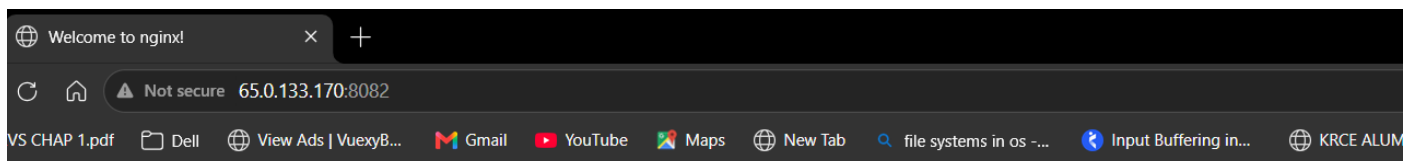
Here I am changing the host port value so that they both can run simultaneously in the same instance.

```
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    93165d051206   2 days ago    161MB
httpd         latest    b9097c67d4a2   2 days ago    117MB
[root@ip-172-31-45-201 ec2-user]# docker run -itd --name containernginx -p 8082:80 nginx:latest
f183ee4be39014e99b1c84d348a608f24eac62dad8eb7126c15b227027d2492
[root@ip-172-31-45-201 ec2-user]# s
```

We can as it starts to run using **ps command**.

```
e67af5af8ac1  httpd:latest    httpd foreground    6 minutes ago    Up 6 minu
[root@ip-172-31-45-201 ec2-user]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS
f183ee4be390   nginx:latest   "/docker-entrypoint...." 51 seconds ago   Up 50 sec
e67af5af8ac1   httpd:latest   "httpd-foreground"       7 minutes ago    Up 7 minu
[root@ip-172-31-45-201 ec2-user]#
```

Copy and paste the public ip, followed by 8082(host port). We can see the output of nginx server.



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Step no:7 To stop the running containers: **docker stop container_ID/container_name**

The id and name of the container can be obtained from **docker ps**.

docker stop container_ID

```
e67af5af8ac1  httpd:latest    httpd foreground    6 minutes ago    Up 6 minu
[root@ip-172-31-45-201 ec2-user]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS
f183ee4be390   nginx:latest   "/docker-entrypoint...." 51 seconds ago   Up 50 sec
e67af5af8ac1   httpd:latest   "httpd-foreground"       7 minutes ago    Up 7 minu
[root@ip-172-31-45-201 ec2-user]# docker stop ^C
[root@ip-172-31-45-201 ec2-user]# docker stop e67af5af8ac1
e67af5af8ac1
```

docker stop container_name

```
[root@ip-172-31-45-201 ec2-user]# docker stop containernginx
containernginx
[root@ip-172-31-45-201 ec2-user]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Step no:8 To remove the containers, first stop it like above and followed by,

docker rm container-1-ID container-2-ID

```
[root@ip-172-31-45-201 ec2-user]# docker rm e67af5af8ac1 f183ee4be390
e67af5af8ac1
f183ee4be390
[root@ip-172-31-45-201 ec2-user]# docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

```
[root@ip-172-31-45-201 ec2-user]#
```

Step no:9 The images will be still there. In order to remove the images

docker rmi container_name:tag

eg: docker rmi httpd:latest

we can also delete multiple images in a single line by adding them with space.

```
CONTAINER ID    IMAGE    COMMAND    CREATED    STATUS    PORTS    NAMES
[root@ip-172-31-45-201 ec2-user]# docker rmi nginx:latest httpd:latest
Untagged: nginx:latest
Untagged: nginx@sha256:800e7c98538c6bf725f5177e841aa720ae0ed1c378bbea368b6330bfe18a
Deleted: sha256:93165d051206cc2ee6804a233effad28ed3db70faba8f86ee5b4ea770e11d6ac
Deleted: sha256:c4b65df849de4dbeb1eb09f4c0b801b34673e68a5a3bac2aefb77aa2353344b2
Deleted: sha256:cb00327f7b9a9e1a43d84ae3066c9d7f9d31a6041393b634becf124de92f3fee
Deleted: sha256:91a24c04c7fe16ecfd786d20e004e8d96b3c545b95ea2130e053ac215adf31ce
Deleted: sha256:f49a4a738b595c01ddab42dc5f47a49a7beff5f72db8437e4270337b139984a6
Deleted: sha256:01e0271b1e8f639b0041c98a1bf00780b785db2a026406aaff59dab4cfc5bbdd
Deleted: sha256:6920217a830f773b01f9967eac8bfc598ac00824ed993bccfb69c1eeff1de175
Untagged: httpd:latest
Untagged: httpd@sha256:2a7deaaaf357261a1dffc8b8fc725f4aaba2af95fbde1a40a68bca7cb0f03
Deleted: sha256:b9097c67d4a2c6d267e015e6297a61afdd1870f904f9229fce3c068db860cd68
Deleted: sha256:6e463bbf010f913986952d453ddb9b9ccc6156c38b45299d4cf54ad07294234a
Deleted: sha256:e2940635c5b95917ffa196896a67a50a0c7ac739722db71591bef27be119802e
Deleted: sha256:a845eb309d6f115a9e9d7d2a100f82b6867ef8b0572412ed8db6188f4ee537b7
Deleted: sha256:2e5669539e9ee3c7437ef60680d1c01e24013dceb5f48d2dee84c5584aaf32be
Deleted: sha256:0b48ec94efabe9e8ecfaef9edfa37d1820260172b7765b6c6494ab252c755fb9
Deleted: sha256:219a998c60509502b47b97f1158067d5dd62640d2d689560d32cfd5594f6bc40
[root@ip-172-31-45-201 ec2-user]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
------------	-----	----------	---------	------